

Contents

Introduction	2
Parallelization Strategy	3
MPI work distribution	4
CUDA kernel design	5
1. Cluster average kernel	5
2. Row-Distance and Row-Label Update Kernels.....	6
3. Column update kernel	6
Bonus Implementation	7
1. OpenMP Acceleration in MPI.....	8
2. Heterogeneous CPU + GPU Execution	9
3. CUDA Streams	10
4. Asynchronous MPI Communication.....	10
5. Warp-Shuffle Kernel.....	11
6. Mixed Precision Computation.....	12
7. Auto-Tuning	13
8. Auto-Tuner Script.....	14
9. Multiple MPI Processes Sharing One GPU	15
Experimental Setup.....	16
Results.....	17
Discussion.....	18
Conclusion.....	19

Introduction

Co-clustering is a data mining technique that simultaneously groups rows and columns of a matrix into clusters based on similarity. Unlike traditional clustering approaches that cluster only one dimension of the data, co-clustering identifies correlated structures between rows and columns, making it useful for applications such as text mining, recommendation systems, and biological data analysis. However, the iterative nature of the algorithm and the large size of modern datasets make co-clustering computationally expensive when executed sequentially.

The goal of this assignment is to develop a scalable high-performance implementation of the co-clustering algorithm using distributed-memory and GPU parallelism. The application was first parallelized using MPI to distribute the workload across multiple compute nodes. To further accelerate computation, CUDA was integrated to offload the most computationally intensive parts of the algorithm to GPUs, resulting in a hybrid MPI+CUDA implementation. In the final version, OpenMP was additionally incorporated to utilize the available CPU cores within each MPI process, creating a heterogeneous MPI + OpenMP+ CUDA implementation.

The dataset is partitioned column-wise across MPI processes, allowing each process to independently operate on a subset of the matrix while synchronizing global cluster statistics through MPI collective communication. Within each MPI process, CUDA kernels accelerate cluster-average computation, row-distance evaluation, row-label updates, and column-label updates. This design combines inter-node parallelism through MPI, shared-memory parallelism through OpenMP, and fine-grained GPU acceleration through CUDA.

Several bonus optimizations were implemented and evaluated on top of the baseline MPI+CUDA implementation. These include asynchronous MPI collective communication, CUDA streams for overlapping GPU operations, warp-shuffle based row-label updates, mixed-precision computation with Kahan compensated summation, configurable CUDA thread block dimensions with automatic tuning, and experiments involving multiple MPI processes sharing a single GPU. Additional kernel-level optimizations such as the use of restrict pointers, loop unrolling, local caching, and reduced indexing overhead were also applied to improve execution efficiency.

Correctness was verified using the Normalized Mutual Information (NMI) metric by comparing generated labels against the provided reference solutions. During development, several issues identified through feedback were corrected, including cluster-average computation inconsistencies and row-label update behavior. After these corrections, all evaluated datasets achieved an NMI score of 1.0. Performance measurements, scalability experiments, and optimization studies were conducted on the DAS6 cluster to evaluate both the baseline implementation and the effectiveness of the proposed bonus optimizations.

Parallelization Strategy

The co-clustering application is parallelized using a heterogeneous MPI + OpenMP + CUDA approach, combining distributed-memory parallelism across compute nodes, shared-memory parallelism within each node, and GPU acceleration inside each MPI process. The workload is distributed across MPI ranks by partitioning the input matrix column-wise, allowing each process to operate independently on a subset of the columns while maintaining globally synchronized clustering information.

The root MPI process reads the complete input matrix and distributes the matrix columns across all ranks using `MPI_Scatterv`. Column labels are partitioned in the same way to ensure that each process owns the labels corresponding to its local columns. A block distribution strategy is used, where the columns are divided as evenly as possible among MPI processes, ensuring good load balancing even when the number of columns is not perfectly divisible by the number of ranks. The local matrix partitions are stored in column-major format to improve memory locality and coalesced memory access patterns on the GPU.

Within each MPI process, CUDA kernels accelerate the most computationally intensive parts of the algorithm. The cluster-average computation, row-distance computation, row-label update phase, and column-label update phase are executed on the GPU. OpenMP is additionally used in the MPI-only implementation to exploit the available CPU cores and provide thread-level parallelism within each MPI process. This heterogeneous design allows the application to exploit inter-node parallelism through MPI, shared-memory parallelism through OpenMP, and fine-grained data parallelism through CUDA.

Global synchronization is required during the computation of cluster statistics such as cluster sums and counts. Each MPI process computes partial sums locally on the GPU, after which the partial results are combined across all processes using asynchronous collective communication through `MPI_Allreduce`. The globally reduced sums and counts are then used to compute consistent cluster averages across all MPI ranks. By using non-blocking collectives, communication overhead can be partially overlapped with computation.

The row-label update phase also requires global synchronization because each process computes partial row-distance contributions based only on its local columns. These partial distances are combined across all MPI processes using `MPI_Allreduce`, after which the final row-label updates are performed on the GPU. The updated row labels are then broadcast to all MPI processes using `MPI_Bcast` to ensure consistency before the next iteration begins.

To further improve GPU utilization, CUDA streams were introduced to allow independent GPU operations to execute concurrently. Separate CUDA streams were used for selected computation phases, reducing idle GPU periods and improving overall throughput. CUDA kernel launches are validated using `cudaGetLastError()` and synchronized using `cudaDeviceSynchronize()` to ensure correctness and simplify debugging. Additionally, MPI failures are handled using `MPI_Abort` to ensure correct termination of all distributed processes in case of runtime errors.

Overall, the heterogeneous MPI + OpenMP + CUDA design minimizes redundant computation, distributes the workload effectively across nodes, utilizes both CPU and GPU resources, and accelerates the computational bottlenecks of the co-clustering algorithm.

MPI work distribution

The workload is distributed across MPI processes using a one-dimensional block decomposition of the matrix columns. Each MPI rank receives a contiguous subset of columns from the original input matrix, along with the corresponding column labels. This column-wise partitioning strategy was chosen because the number of columns in the datasets is significantly larger than the number of rows, allowing the workload to be distributed more evenly across MPI processes while exposing substantial parallelism.

The total number of columns is divided approximately equally among the available MPI ranks. When the number of columns is not perfectly divisible by the number of processes, the remaining columns are assigned to the lower-ranked processes to maintain balanced work distribution. The root process computes the send counts and displacements required for `MPI_Scatterv`, which is used to distribute both the matrix data and the column labels across all nodes.

Each MPI process performs computations independently on its local matrix partition. During each iteration, local cluster sums and counts are computed using only the columns assigned to the process. These partial results are then globally aggregated using MPI collective communication to produce globally consistent cluster statistics. Similarly, each process computes partial row-distance contributions using its local columns, after which the globally reduced distances are used to update row labels.

The communication pattern is dominated by collective operations, primarily `MPI_Allreduce` and `MPI_Bcast`. Asynchronous reductions were introduced to reduce synchronization overhead and allow communication to overlap with useful computation whenever possible. Compared with the original blocking `MPI_Allreduce` implementation, the non-blocking approach improved scalability while preserving correctness. The communication overhead becomes more noticeable at larger node counts because the amount of computation per node decreases while the synchronization cost remains relatively constant.

The column-wise decomposition strategy proved effective for scalability because the computational cost of column-label updates and cluster-sum computation scales directly with the number of columns assigned to each process. Furthermore, since column-label updates are performed independently on each process after synchronization, no additional communication is required during the column-label update phase, reducing communication overhead during later stages of the iteration.

As an additional experiment, multiple MPI processes were configured to share a single GPU. This was achieved by restricting the visible CUDA devices and executing multiple MPI ranks on the same accelerator. Although correctness was preserved (NMI = 1.0), the experiments showed that GPU contention increased execution time, demonstrating the trade-off between increased process-level parallelism and shared accelerator resources.

CUDA Kernel design

Within each MPI process, GPU acceleration is used to parallelize the most computationally intensive stages of the co-clustering algorithm. The CUDA implementation is organized into separate kernels for cluster-average computation, row-distance computation, row-label updates, and column-label updates. Each kernel is designed to exploit data parallelism while minimizing synchronization overhead and maintaining correct clustering behavior across iterations.

The kernels use a one-dimensional thread mapping strategy, where threads are assigned either to matrix rows or matrix columns depending on the operation being performed. Column-based kernels expose significantly more parallelism because the datasets contain a very large number of columns, while row-based kernels are limited by the relatively small number of rows. CUDA kernels execute asynchronously, and selected operations are executed in separate CUDA streams to improve concurrency. Explicit synchronization is performed when required using `cudaDeviceSynchronize()` to ensure correctness and enable reliable timing measurements. Kernel launch failures are validated using `cudaGetLastError()` for improved debugging and robustness.

Several CUDA optimization techniques were implemented in the final version of the application. These include the use of restrict pointers, loop unrolling, local caching of frequently accessed values, reduced indexing overhead, warp-shuffle reductions, mixed-precision computation with compensated summation, and configurable CUDA thread block sizes selected through auto-tuning experiments.

1. Cluster average kernel

The cluster-average computation is implemented using the `calculate_local_sums_kernel`. Each CUDA thread is assigned to a local matrix column and iterates sequentially over all rows of that column. For every matrix element, the thread determines the corresponding row and column cluster labels and accumulates partial cluster sums and counts. Since multiple threads may concurrently update the same cluster entries, `atomicAdd` operations are used to ensure correctness when updating shared cluster-sum and count arrays in global memory.

Several optimizations were introduced in the final implementation of this kernel. Repeated indexing arithmetic was reduced through local caching of column offsets and frequently used values, while restrict pointers improve compiler alias analysis and memory optimization opportunities. These changes reduce instruction overhead and improve memory-access efficiency during repeated kernel execution. Because each thread processes an independent matrix column, the kernel exposes substantial parallelism and scales effectively on larger datasets.

2. Row-Distance and Row-Label Update Kernels

The row-distance computation is implemented in the `compute_row_dist_parallel_kernel`. Each thread processes a local matrix column and accumulates distance contributions for all row clusters. The resulting row-distance matrix is later combined across MPI processes using collective communication. Compared with earlier versions of the implementation, this approach exposes significantly more GPU parallelism because work is distributed across columns rather than assigning only one thread per row.

The row-label update phase is implemented using two kernel variants. The baseline version assigns one thread to each row and selects the cluster with the minimum distance. An optimized variant, `update_row_labels_kernel_warp`, uses CUDA warp-shuffle instructions (`__shfl_down_sync`) to perform reductions within a warp. This approach reduces reduction overhead and serves as an alternative CUDA reduction strategy. Experimental evaluation showed that the warp-shuffle implementation preserved correctness (NMI = 1.0) while demonstrating the use of warp-level communication primitives.

Additional optimizations include local caching of frequently accessed values, reduced indexing overhead, and loop unrolling to improve instruction-level parallelism and GPU efficiency.

3. Column update kernel

The column-label update phase is implemented using the `update_col_kernel`. In this kernel, each thread processes one local matrix column and computes its distance to all possible column clusters using the current row labels and cluster averages. The thread then selects the cluster with the minimum distance and updates the column label if necessary. Because the number of columns is significantly larger than the number of rows, this kernel exposes substantial parallelism and achieves high GPU occupancy.

A mixed-precision variant, `update_col_kernel_fp32`, was also implemented as a bonus optimization. This version performs distance accumulation using single-precision arithmetic while employing Kahan compensated summation to reduce numerical error. Experimental evaluation showed that the mixed-precision implementation maintained NMI = 1.0 while reducing execution time from 2.58 seconds to 1.46 seconds, corresponding to a speedup of approximately 1.77 $\times$. Atomic operations are used only for counting label changes and accumulating global distance values, minimizing synchronization overhead during the update phase.

The final implementation also supports configurable CUDA thread block sizes through a compile-time `BLOCK_SIZE` parameter. Multiple block sizes were evaluated using an auto-tuning framework, with 64 threads per block providing the best performance on the target GPU platform.

Bonus Implementation

In addition to the baseline MPI+CUDA implementation, several bonus optimizations and experimental extensions were implemented and evaluated to improve performance, increase hardware utilization, and explore advanced parallel programming techniques. To preserve the original implementation, all bonus features were implemented in a separate source file (`cgc_cuda_bonus.cu`), allowing direct benchmarking against the baseline CUDA version while maintaining correctness verification throughout development.

The final bonus implementation combines optimizations at multiple levels of the system. At the MPI level, asynchronous collective communication using `MPI_Allreduce` was introduced to reduce synchronization overhead and overlap communication with computation. An additional OpenMP-enhanced MPI implementation was developed to utilize multiple CPU cores within each MPI process. Together with CUDA acceleration, this resulted in a heterogeneous MPI + OpenMP + CUDA implementation capable of exploiting parallelism across distributed nodes, CPU cores, and GPUs simultaneously.

Several GPU-specific optimizations were also implemented. CUDA streams were used to execute independent GPU operations concurrently, improving device utilization and reducing idle periods. Alternative CUDA kernel variants were developed, including a warp-shuffle row-label update kernel that uses `__shfl_down_sync` to perform warp-level reductions efficiently. A mixed-precision column-update kernel was implemented using single-precision arithmetic combined with Kahan compensated summation, significantly reducing execution time while preserving clustering accuracy.

Kernel-level optimizations included the use of restrict pointers, loop unrolling, local caching of frequently accessed values, reduced indexing overhead, and configurable CUDA thread block dimensions. An auto-tuning framework was developed to evaluate multiple block sizes and automatically select the best-performing configuration for the target hardware. A companion tuning script was implemented to automate compilation, execution, and performance comparison across candidate configurations.

An additional experiment investigated the effect of allowing multiple MPI processes to share a single GPU. Although correctness was preserved (NMI = 1.0), the experiments showed that GPU contention increased execution time, demonstrating the trade-off between increased process-level parallelism and limited accelerator resources.

All bonus optimizations preserved clustering correctness and achieved an NMI score of 1.0 on the evaluated datasets. Performance measurements demonstrated that several optimizations provided substantial benefits, with the mixed-precision implementation reducing execution time from 2.58 seconds to 1.46 seconds while maintaining identical clustering results. These experiments illustrate the effectiveness of combining communication optimizations, CPU parallelism, GPU acceleration, and kernel-level tuning techniques within a single high-performance co-clustering application.

1. OpenMP Acceleration in MPI

The MPI-only implementation was extended using OpenMP to utilize all CPU cores assigned to each MPI rank. OpenMP parallel for directives were added to computationally intensive loops, enabling thread-level parallelism within each MPI process. This approach complements the distributed-memory parallelism provided by MPI by exploiting the shared-memory parallelism available on each compute node.

Experiments were performed using OMP_NUM_THREADS=16, corresponding to the number of CPU cores allocated to each MPI rank. The objective was to improve CPU utilization and reduce the amount of idle processor time during local computations. The OpenMP implementation preserved clustering correctness and produced identical clustering results to the original MPI implementation.

The below table presents the correctness comparison between the original MPI implementation and the OpenMP-enhanced version. Both implementations achieved identical clustering quality with an NMI score of 1.0, confirming that the introduction of thread-level parallelism did not affect the algorithmic behavior of the application.

Configuration	NMI
MPI	1.00
MPI + OpenMP	1.00

The results demonstrate that OpenMP can be successfully combined with MPI to increase CPU utilization while maintaining clustering correctness. This extension forms the CPU component of the final heterogeneous MPI + OpenMP + CUDA implementation.

2. Heterogeneous CPU + GPU Execution

A heterogeneous implementation combining MPI, CUDA, and OpenMP was developed to exploit multiple levels of parallelism within the system. MPI is responsible for distributing the matrix columns across compute nodes, CUDA accelerates the most computationally intensive stages of the co-clustering algorithm on the GPU, and OpenMP utilizes the available CPU cores within each MPI process. This hybrid design enables the application to leverage distributed-memory parallelism, shared-memory parallelism, and GPU acceleration simultaneously.

The computationally intensive phases, including cluster-average computation, row-distance computation, row-label updates, and column-label updates, are executed on the GPU through CUDA kernels. MPI handles the exchange of global clustering statistics and synchronization between distributed processes, while OpenMP accelerates CPU-side computations and improves utilization of the processor resources assigned to each MPI rank.

By combining these three programming models, the implementation is able to exploit the strengths of each hardware component. GPUs provide massive data parallelism for matrix computations, OpenMP improves CPU utilization within each node, and MPI enables the application to scale across multiple nodes. The resulting heterogeneous design forms the final optimized implementation used for performance evaluation.

Correctness was verified by comparing the generated clustering labels with the provided reference solutions. The heterogeneous implementation preserved clustering accuracy and achieved an NMI score of 1.0 on the evaluated datasets.

3. CUDA Streams

CUDA streams were introduced to increase GPU utilization by allowing independent GPU operations to execute concurrently whenever possible. In the baseline implementation, GPU kernels were executed sequentially, causing certain operations to wait for previous kernels to complete before new work could begin. This approach can leave portions of the GPU underutilized, particularly when independent computations are available.

To address this limitation, separate CUDA streams were used for selected stages of the co-clustering algorithm. In particular, independent operations such as cluster-average computation and row-distance computation were assigned to different streams. This allowed the GPU scheduler to overlap kernel execution and memory operations when hardware resources permitted, reducing idle periods and improving overall throughput.

The use of CUDA streams required careful synchronization to ensure correctness between dependent stages of the algorithm. Synchronization points were inserted only when necessary, allowing independent GPU work to proceed concurrently while preserving the correctness of the clustering process.

Experimental evaluation showed that the stream-based implementation maintained clustering correctness and achieved an NMI score of 1.0. Although the performance improvement depended on the workload characteristics and available GPU resources, the use of CUDA streams improved resource utilization and demonstrated the potential benefits of overlapping independent GPU operations within a hybrid MPI+CUDA application.

4. Asynchronous MPI Communication

The baseline implementation relied on blocking collective communication using `MPI_Allreduce` to combine partial cluster statistics and row-distance contributions across all MPI processes. Although straightforward to implement, blocking collectives force all processes to wait until the communication completes before continuing execution, introducing synchronization overhead and reducing parallel efficiency. To reduce this overhead, blocking `MPI_Allreduce` operations were replaced with asynchronous collective communication using `MPI_Iallreduce`. Non-blocking collectives allow communication to be initiated while computation continues, enabling communication and computation to overlap whenever independent work is available. This approach reduces idle waiting time and improves the utilization of distributed resources.

The asynchronous implementation required explicit synchronization through `MPI_Wait` before the reduced data could be used in subsequent computation stages. By delaying synchronization until the communication results were actually needed, the application was able to reduce the impact of communication latency while maintaining correct behavior.

Experimental evaluation confirmed that the asynchronous communication strategy preserved clustering correctness and achieved an NMI score of 1.0 on the evaluated datasets. The

optimization demonstrates how overlapping communication and computation can improve the efficiency of distributed-memory applications without modifying the underlying clustering algorithm.

5. Warp-Shuffle Kernel

An alternative row-label update kernel was implemented using CUDA warp-shuffle instructions. The baseline implementation assigns one thread to each row and determines the cluster with the minimum distance by iterating through all candidate row labels. To explore a more advanced CUDA reduction strategy, a second kernel variant was developed using warp-level communication primitives.

The optimized implementation uses the CUDA intrinsic `__shfl_down_sync` to exchange values directly between threads within a warp. Instead of relying on global memory or explicit synchronization, partial minimum-distance values are propagated through the warp using warp-shuffle operations until the minimum distance and corresponding cluster label are identified. This approach reduces reduction overhead and exploits the fast communication capabilities available within a CUDA warp.

The warp-shuffle implementation was integrated as an alternative row-label update kernel and benchmarked against the baseline version. Experimental evaluation confirmed that the optimized kernel preserved clustering correctness and produced identical clustering results, achieving an NMI score of 1.0 on the evaluated datasets.

The implementation demonstrates the use of warp-level communication primitives as an alternative reduction strategy in CUDA. While the overall performance improvement depends on the workload characteristics and the size of the reduction, the warp-shuffle kernel reduces synchronization requirements and illustrates a more advanced GPU optimization technique compared with conventional reduction approaches.

Kernel variant	NMI
Baseline row update	1.00
Warp-shuffle row update	1.00

6. Mixed Precision Computation

A mixed-precision variant of the column-update kernel was implemented to reduce computational cost while maintaining clustering accuracy. In the baseline implementation, distance accumulation is performed using double-precision arithmetic. Although numerically robust, double-precision operations require more memory bandwidth and computational resources than single-precision arithmetic on modern GPUs.

To improve performance, an alternative kernel was developed that performs distance accumulation using single-precision floating-point arithmetic. To compensate for the potential loss of numerical accuracy, Kahan compensated summation was incorporated into the accumulation process. This technique reduces floating-point rounding errors by maintaining a compensation term during repeated additions, improving numerical stability compared with naive single-precision accumulation.

The mixed-precision implementation was evaluated against the original double-precision version. Experimental results showed that clustering correctness was fully preserved, with both implementations achieving an NMI score of 1.0 on the evaluated datasets. At the same time, the mixed-precision kernel significantly reduced execution time by decreasing arithmetic and memory-access costs on the GPU.

The table below presents the performance comparison between the baseline and mixed-precision implementations.

Implementation	Clustering Time (s)	NMI
Double precision (fp64)	2.5841	1.0
Mixed precision (fp32 + Kahan)	1.46243	1.0

The mixed-precision implementation achieved a speedup of approximately 1.77× while maintaining identical clustering results. This demonstrates that carefully designed mixed-precision techniques can provide substantial performance improvements without sacrificing clustering quality.

7. Auto-Tuning

To investigate the impact of CUDA thread block dimensions on performance, a configurable compile-time `BLOCK_SIZE` parameter was introduced into the CUDA implementation. Different thread block sizes can significantly influence GPU occupancy, memory-access behavior, and overall kernel efficiency. Rather than selecting a block size arbitrarily, an auto-tuning approach was used to identify the best-performing configuration for the target hardware and workload.

Several candidate block sizes were evaluated by recompiling the application with different `BLOCK_SIZE` values and measuring the clustering runtime. The experiments were performed using the optimized CUDA implementation while maintaining identical clustering parameters and datasets. Correctness was verified after each experiment, and all configurations preserved an NMI score of 1.0.

The table below summarizes the measured performance for the evaluated block sizes.

Block Size	Clustering time (s)
64	2.13793
128	2.15271
256	3.45588
512	3.79084

The results show that smaller thread block sizes provided the best performance for this application. A block size of 64 threads achieved the lowest runtime and was therefore selected as the default configuration for subsequent experiments. Larger block sizes increased execution time, likely due to reduced scheduling flexibility and less efficient utilization of GPU resources for the workload characteristics of the co-clustering kernels.

The auto-tuning experiments demonstrate the importance of parameter tuning in GPU applications and show that a simple configuration change can have a noticeable impact on overall performance

8. Auto-Tuner Script

To automate the parameter-tuning process, a tuning script (`tune.sh`) was developed to compile and benchmark multiple CUDA configurations automatically. Rather than manually recompiling the application for each candidate thread block size, the script iterates through a predefined set of `BLOCK_SIZE` values, recompiles the CUDA implementation, executes the clustering application, and records the resulting execution times.

The script was designed to evaluate several candidate configurations, including block sizes of 64, 128, 256, and 512 threads per block. By automating the compilation and benchmarking workflow, the tuning process became faster, more reproducible, and less prone to human error. This approach also simplifies portability because the same script can be executed on different GPU architectures to identify the most suitable configuration for the available hardware.

After all benchmark runs are completed, the recorded execution times can be compared to determine the best-performing configuration. In the experiments performed for this project, the script identified a block size of 64 threads as the optimal setting for the evaluated dataset and GPU platform.

The auto-tuner script complements the block-size auto-tuning framework by providing an automated mechanism for performance exploration. This allows kernel parameters to be optimized systematically and enables reproducible performance studies across different datasets and hardware configurations.

9. Multiple MPI Processes Sharing One GPU

This experiment was performed to investigate the effect of allowing multiple MPI processes to share a single GPU. This configuration was implemented by restricting the visible CUDA devices using the `CUDA_VISIBLE_DEVICES` environment variable and launching multiple MPI ranks on the same accelerator. The objective was to determine whether smaller GPU workloads distributed across multiple MPI processes could improve overall performance.

Two configurations were evaluated. The baseline configuration used a single MPI process with exclusive access to one GPU, while the experimental configuration executed four MPI processes sharing the same GPU. In both cases, the clustering results were compared against the provided reference solution to verify correctness.

The below table summarizes the performance results obtained during the experiments.

Configuration	Clustering time (s)	NMI
1 MPI Rank, 1 GPU	2.5841	1.0
4 MPI Rank, 1 GPU	4.6278	1.0

The results show that clustering correctness was fully preserved, with both configurations achieving an NMI score of 1.0. However, sharing the GPU among multiple MPI processes increased execution time by approximately 79%, resulting in a slowdown of about 1.8× compared with the baseline configuration.

This behavior indicates that GPU contention and additional process-management overhead outweighed any benefit obtained from dividing the workload among multiple MPI ranks. Although GPU sharing did not improve performance for this application, the experiment demonstrates the practical trade-offs involved when multiple distributed processes compete for the same accelerator resources.

Experimental Setup

All experiments were performed on the DAS6 cluster using the hybrid MPI+CUDA implementation of the co-clustering algorithm. MPI was used for distributed execution across nodes, while CUDA accelerated local computations on GPUs. One MPI process and one GPU were used per node.

The applications were compiled using OpenMPI 4.1.6 and CUDA Toolkit 12.1 with `-O3` optimization enabled. Performance experiments were mainly conducted on the medium dataset using 1, 2, 4, and 8 MPI nodes to evaluate scalability. The XL and XXL datasets were additionally used for correctness validation on larger problem sizes.

Correctness was verified using the normalized mutual information (NMI) metric by comparing generated labels with the provided reference solutions. All final CUDA+MPI evaluations achieved an NMI score of 1.0. The optimized CUDA bonus implementation was also benchmarked using different CUDA thread block sizes to evaluate the effect of launch configuration on runtime performance

Experimental Configuration

Component	Configuration
Cluster System	DAS6
MPI Library	OpenMPI 4.1.6
CUDA Toolkit	CUDA 12.1
MPI Processes	1, 2, 4, 8
Datasets	m, xl, xxl
Maximum Iterations	100
CUDA Block Sizes Tested	64, 128, 256, 512
Correctness Metric	NMI

Results

The correctness of the final implementation was verified using the Normalized Mutual Information (NMI) metric. During development, several issues identified through the feedback process were corrected, including cluster-average computation inconsistencies and row-label update behavior. After these corrections, all evaluated implementations achieved an NMI score of 1.0, demonstrating that the introduced optimizations preserved the clustering behavior of the original algorithm.

The bonus optimizations were evaluated to determine their impact on performance and scalability. OpenMP was successfully integrated into the MPI implementation, allowing multiple CPU cores to be utilized within each MPI process. In combination with CUDA acceleration, this resulted in a heterogeneous MPI + OpenMP + CUDA implementation capable of exploiting distributed-memory parallelism, shared-memory parallelism, and GPU acceleration simultaneously. Additional communication optimizations were introduced through asynchronous MPI collectives, while CUDA streams enabled overlapping GPU operations and improved device utilization.

Among all evaluated optimizations, mixed-precision computation provided the largest measurable performance improvement. By replacing double-precision distance accumulation with single-precision arithmetic combined with Kahan compensated summation, the clustering runtime was reduced from 2.58 seconds to 1.46 seconds while maintaining an NMI score of 1.0. This corresponds to a speedup of approximately 1.77× and demonstrates the effectiveness of mixed-precision techniques for GPU-accelerated scientific applications.

The auto-tuning framework showed that kernel launch configuration has a significant impact on performance. Multiple thread block sizes were evaluated automatically, and a block size of 64 threads provided the best performance on the target GPU platform. The accompanying tuning script automated compilation and benchmarking, allowing the most efficient configuration to be selected systematically and reproducibly.

An additional experiment investigated the effect of allowing multiple MPI processes to share a single GPU. Although correctness remained unchanged and an NMI score of 1.0 was maintained, execution time increased when four MPI ranks shared the same GPU. The observed slowdown indicates that GPU contention outweighed any benefit obtained from smaller per-rank workloads. Overall, the results demonstrate that combining MPI, OpenMP, CUDA, communication optimizations, kernel tuning, and mixed-precision computation can significantly improve the performance of the co-clustering application while preserving clustering accuracy.

Discussion

The results demonstrate that the co-clustering algorithm benefits significantly from combining distributed-memory parallelism, shared-memory parallelism, and GPU acceleration. The MPI implementation enables the workload to be distributed across multiple nodes, while CUDA accelerates the most computationally intensive stages of the algorithm. The addition of OpenMP further improves resource utilization by allowing multiple CPU cores to contribute to the computation within each MPI process. Together, these techniques form an effective heterogeneous computing solution capable of exploiting the available hardware at multiple levels.

Several optimization techniques were evaluated as part of the bonus implementation. Communication overhead was reduced through asynchronous MPI collectives, while CUDA streams improved GPU utilization by enabling overlapping operations. The warp-shuffle row-label update kernel demonstrated the use of efficient warp-level communication primitives, and the auto-tuning framework showed that kernel launch parameters can have a substantial impact on performance. These optimizations improved the efficiency of the implementation without affecting clustering correctness.

Among all evaluated optimizations, mixed-precision computation provided the largest measurable performance improvement. The combination of single-precision arithmetic and Kahan compensated summation significantly reduced execution time while maintaining an NMI score of 1.0. This result highlights the importance of balancing numerical precision and computational efficiency in GPU-accelerated applications.

The experiment involving multiple MPI processes sharing a single GPU provided an interesting insight into resource contention. Although correctness was preserved, performance decreased because multiple processes competed for the same accelerator resources. This demonstrates that increasing the degree of parallelism does not necessarily improve performance when hardware resources become a bottleneck. Overall, the project illustrates the trade-offs involved in designing scalable high-performance applications and the importance of evaluating optimization techniques experimentally.

Conclusion

In this project, a high-performance co-clustering application was developed using a hybrid MPI+CUDA approach and subsequently extended into a heterogeneous MPI + OpenMP + CUDA implementation. The matrix was distributed across MPI processes using column-wise decomposition, while CUDA kernels accelerated the computationally intensive stages of the algorithm. OpenMP was integrated to utilize multiple CPU cores within each MPI process, enabling the application to exploit distributed-memory, shared-memory, and GPU parallelism simultaneously.

Several advanced optimizations were implemented and evaluated, including asynchronous MPI communication, CUDA streams, warp-shuffle kernels, mixed-precision computation, auto-tuning of CUDA thread block sizes, and experiments involving multiple MPI processes sharing a single GPU. Throughout development, correctness was continuously verified using the Normalized Mutual Information metric, and all final implementations achieved an NMI score of 1.0 after addressing issues identified through feedback and validation.

The experimental results showed that the implemented optimizations can significantly improve performance while preserving clustering accuracy. In particular, mixed-precision computation achieved the largest performance gain, reducing execution time by approximately 1.77× without affecting the clustering results. The project demonstrates how multiple parallel programming techniques and optimization strategies can be combined effectively to accelerate a complex iterative algorithm on modern heterogeneous computing systems.